

s3.05 — Distributed lease / lock service — leader election and fencing

Index

- Interview framing
- Requirements — IC3 baseline
- Capacity and consequence — IC3 baseline
- Architecture and flow — IC3 baseline
- API and data model — IC3 baseline
- Acquire, renew, and uncertain ownership — IC3 baseline
- Resource fencing and recovery epochs — IC3 baseline
- Failure, operations, and security
- Alternatives and IC4 stretch
- Level signal — IC3 baseline vs IC4 stretch
- Interview pushback and validation
- Recall card
- References

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC3 baseline is what must land in every phase.
 - IC4 stretch changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: ranked bank.
 - Prior supplemental bank and recruiter concurrency concepts.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: write a lock key with a TTL and assume the former holder stops when it expires.
 - First break: a paused holder resumes after a new owner acquires the lock and overwrites the protected resource.
 - Second break: a renewal times out so the holder cannot tell whether it still owns the lease, and a snapshot restore of the authority hands out a token lower than one a resource already accepted.
 - Minimum primitives
 - Linearizable lease ownership.
 - Monotonic fencing epochs.
 - Atomic fence enforcement at the resource.
 - Conservative renewal and uncertain-outcome handling.
- 🗣️ "A lock controls ownership records; fencing controls what an old owner can still change."
- Opening script
 - "I will design a leased lock whose safety is enforced by the protected resource."
 - "I will first ask whether that resource can reject stale fencing tokens."
 - "The lock authority will use a quorum-backed state machine for acquisition and renewal."
 - "The deep dive is a paused holder resuming after expiry and a new owner establishing its fence."
 - "I will close with uncertain renewals, fair waiting, and restoring token monotonicity."

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt

Phase	Clock	Deliverable
Requirements	05:00–11:00	Scope and one correctness invariant
Capacity and API	11:00–17:00	A number that forces a choice; retry contract
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
What resource is protected?	Metadata writes supporting atomic fencing checks	A lock alone cannot protect arbitrary side effects
Failure model?	Crash, pause, partition, and delayed messages	TTL does not imply process termination
Lease duration?	30 s; heartbeat every 10 s	Trade recovery speed against renewal sensitivity
Fairness?	Best effort by default	Strict FIFO requires a durable waiter queue
Scope?	Regional ownership; one authority per lock	Cross-region authority adds latency and partition coupling

- Functional scope
 - Acquire a named lease.
 - Renew or release only the matching owner and epoch.
 - Return a fencing token to the holder.
 - Wait for ownership changes.
 - Recover expired owners safely.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Acquire latency	p99 < 20 ms regional assumption	Quorum-backed lock transaction
Lease TTL	30 s baseline	Authority-managed lease deadline
Renew cadence	Every 10 s	Conservative client timer and jitter
Recovery delay	Target < 40 s after last successful renewal	Expiry detection plus acquisition
Safety	0 stale-epoch committed writes after fence activation	Atomic check-and-effect at protected resource
Availability	99.9% target with 1 replica failure	3-node quorum authority

- Invariants
 - One committed owner exists for a lock at a time.
 - A fencing token never decreases across release, deletion, or recovery.
 - The resource atomically rejects writes below its highest activated epoch.
 - A new holder must activate its fence before treating itself as the effective resource owner.
 - Renew and release compare owner identity, lease ID, and epoch.
- Non-goals

- Stopping a partitioned process through a TTL.
- Protecting a resource that cannot enforce fencing or idempotency.
- Instant failover with no timing or quorum assumptions.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Renewal load
 - Assume 100,000 active locks renewed every 10 s.
 - 10,000 renewals/s reach the authority before acquisitions.
 - Decision: benchmark lease handling and group related resource ownership under one session only when the shared failure domain is acceptable.
- State
 - 1M retained lock/token records at 200 B is about 200 MB before replicas and indexes.
 - Decision: persist epoch continuity even when empty lock rows are garbage-collected.
- Hot lock
 - A lock protecting a 100 ms critical section admits at most roughly 10 sequential sections/s.
 - Decision: shorten the critical section or partition the resource instead of adding lock servers.
- Waiters
 - 10,000 waiters all retrying on release create a thundering herd.
 - Decision: use jittered CAS retries or a predecessor-watch queue when fairness is required.

Architecture and flow — IC3 baseline

```

Client -> quorum lease authority -> owner + lease + epoch
      |               |
      |               | watch hints
      |               |
      +--> protected resource [activate epoch; atomically check fence + effect]

Old paused client ----- stale epoch -----X protected resource
  
```

- Lease authority
 - Buys one ordered ownership decision.
 - Pays quorum latency and unavailable minority behavior.
 - Client lease manager
 - Buys bounded voluntary use of uncertain ownership.
 - Pays heartbeat traffic and conservative early stopping.
 - Protected resource
 - Buys rejection of stale-owner writes.
 - Pays an atomic epoch check on every protected effect.
 - Waiter queue
 - Buys reduced contention or FIFO ordering when needed.
 - Pays durable queue cleanup and watch recovery.
- Acquire through a linearizable compare against absent or expired ownership.
- Receive owner identity, lease ID, TTL, and a new fencing epoch.
- Activate that epoch atomically at the protected resource.
- Perform each effect with the same epoch and an operation identity.
- Renew with the matching owner tuple before the conservative local deadline.
- Release conditionally or let the authority expire the lease.

API and data model — IC3 baseline

- Proposed API contracts.
 - Authenticate the tenant from credentials before looking up any resource.
 - Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
POST /locks/{name}:acquire	Owner nonce, TTL, requestId	Replay returns original acquisition result; held lock returns 409
POST /leases/{id}:renew	Owner, epoch, expected lease	Stale or expired identity is 409/410; uncertainty makes client stop
DELETE /leases/{id}	Matching owner and epoch	Replay is harmless; cannot delete a successor owner
GET /locks/{name}	Owner, epoch, authoritative lease state	A read is observation, not ownership
GET /locks/{name}:watch	Revision cursor	Hint to recheck; compaction requires snapshot

State	Key	Fields and invariant
Lock	(tenant, lockName)	Owner, leaseId, epoch, deadline
Epoch allocator	(scope, recoveryEpoch)	Durable monotonic source independent of empty lock lifetime
Acquisition result	(tenant, client, requestId)	Request hash, lease and epoch, expiry
Resource fence	(tenant, resourceId)	Highest activated epoch
Effect dedup	(resourceId, epoch, operationId)	Committed result with the effect

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Lock rows and acquisition results live in the consensus store as replicated commands; only tenant namespaces, the recovery-epoch allocator, and the resource-side fence and dedup registry are relational.

```

CREATE TABLE lock_namespaces (
  tenant_id      text NOT NULL,
  namespace      text NOT NULL,
  scope          text NOT NULL,
  authority_group text NOT NULL,
  default_ttl_seconds integer NOT NULL DEFAULT 30,
  max_ttl_seconds integer NOT NULL DEFAULT 300,
  lock_quota     integer NOT NULL,
  state          text NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (tenant_id, namespace),
  CHECK (state IN ('ACTIVE', 'SUSPENDED')),
  CHECK (default_ttl_seconds > 0 AND max_ttl_seconds >= default_ttl_seconds AND lock_quota >= 0)
);

CREATE TABLE recovery_epochs (
  scope          text NOT NULL,
  recovery_epoch bigint NOT NULL,
  reason         text NOT NULL,
  snapshot_ref   text,
  activated_at   timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (scope, recovery_epoch),
  allocation_sequence)
  CHECK (reason IN ('INITIAL', 'RESTORE', 'MIGRATION')),
  CHECK (recovery_epoch >= 1)
);
CREATE INDEX recovery_epochs_current ON recovery_epochs (scope, recovery_epoch DESC);

CREATE TABLE resource_fences (
  tenant_id      text NOT NULL,
  resource_id    text NOT NULL,
  highest_epoch  bigint NOT NULL DEFAULT 0,
  resource_version bigint NOT NULL DEFAULT 0,
  activated_at   timestampz,
  PRIMARY KEY (tenant_id, resource_id),
  CHECK (highest_epoch >= 0 AND resource_version >= 0)
);

CREATE TABLE effect_dedup (
  tenant_id      text NOT NULL,
  resource_id    text NOT NULL,
  epoch          bigint NOT NULL,
  operation_id   text NOT NULL,
  result         jsonb NOT NULL,
  committed_at   timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, resource_id, epoch, operation_id),
  FOREIGN KEY (tenant_id, resource_id) REFERENCES resource_fences (tenant_id, resource_id),
  CHECK (epoch >= 0)
);
CREATE INDEX effect_dedup_expiry ON effect_dedup (committed_at);

```

- Not captured by the DDL
 - Lock (tenant, lockName) and Acquisition result (tenant, client, requestId) are replicated state-machine rows; the allocationSequence half of the epoch is allocated inside that ordered command, not by a SQL sequence.
 - Watch revisions, waiter queues, and the client's conservative local deadline are in-memory or log state.
 - resource_fences and effect_dedup commit in the protected resource's database, so no foreign key can reach the lock authority.
- Deviation from the companion
 - The companion distributed_locks table is the SQL-variant authority; this file chooses the consensus variant, so that table is intentionally absent.
 - The companion protected_records row is split into resource_fences (high-water mark) and effect_dedup (per-operation result) to match the state table.

Acquire, renew, and uncertain ownership — IC3 baseline

- Acquisition
 - Linearizably compare lock ownership and authority expiry.
 - Allocate a new epoch in the same ordered state machine command.
 - Do not reset the token when the lock key is deleted.
- Renewal
 - Match owner, lease ID, and epoch.
 - Start the client's conservative deadline from the renewal send time.
 - Subtract a safety margin for local clock behavior and elapsed request time.
 - A timeout leaves renewal uncertain.
 - Stop issuing new work before the local deadline rather than waiting for a definitive rejection.
- Pause boundary
 - A process can pause beyond any chosen TTL.
 - A resumed process must recheck its local deadline.
 - Messages it already sent can still arrive late.
 - Resource-side fencing handles that remaining race.
- Release
 - Delete only if the owner tuple still matches.
 - A stale release must not delete a newly acquired lock.
- Waits
 - Treat watch events as hints and recheck ownership through the authority.
 - Handle disconnects and compacted revision history by resnapshotting.
 - A general key watch does not automatically notify exactly one waiter.

Resource fencing and recovery epochs — IC3 baseline

- Activation boundary
 - The resource stores the highest accepted fencing epoch.
 - A new owner first installs its epoch through an atomic resource operation.
 - An old owner may still write before that activation if the resource has not learned about the newer epoch.
 - Therefore the safety claim begins at resource fence activation, not merely wall-clock lease expiry.

```
ACTIVATE(resource, newEpoch):
    transaction:
        lock resource fence
        require newEpoch >= highestEpoch
        highestEpoch = newEpoch
APPLY(resource, epoch, operationId, effect):
    transaction:
        lock resource fence
        require epoch == highestEpoch
        if committed result exists for (resource, epoch, operationId): return it
        apply effect with any required resource-version comparison
        persist deduplication result in this same transaction
```

- Multiple operations by one owner
 - Equal epochs are valid for multiple distinct operations.
 - Rejecting every `epoch <= highestEpoch` incorrectly rejects the current owner's second write.
 - Use operation IDs for retries.
 - Use expected resource versions or sequence numbers when order within one epoch matters.
- Unsupported destination
 - An email gateway or arbitrary remote API may not support an epoch check.
 - Use a durable effect owner with destination idempotency or admit possible duplicates.

- A lock service cannot manufacture an atomic boundary inside another system.
- Backup restore
 - Restoring an older authority snapshot can lower revision-derived tokens.
 - Preserve a separately monotonic recovery epoch or use a supported revision-bump recovery procedure.
 - A tuple (recoveryEpoch, allocationSequence) can preserve order if the recovery epoch authority is not rolled back.
 - Existing resource fences must survive the same disaster.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Holder pauses beyond TTL	One protected resource	Expired lease and stale-token rejection	New owner activates a higher fence	Reject old effects and reacquire
Renew reply lost	One holder	Renew timeout	Stop new work at conservative deadline	Read/reacquire safely; do not assume renewal failed
Authority quorum lost	New acquisitions and renewals	Quorum errors	No new grants; current holders stop at their conservative deadline	Restore majority; resources retain highest fence
Old release arrives late	One lock	Owner mismatch	No-op or conflict	Current owner continues
Restore lowers allocator	All affected locks	Recovery epoch validation	Block grants	Advance independent recovery epoch before reopening
Client clock runs fast or slow	One holder	Renewal margin violations	Conservative deadline uses elapsed monotonic time, not wall clock	Fix the clock source; the fence catches the remainder
Resource fence store unavailable	All effects on that resource	Activation and apply errors	Holders keep the lease but perform no protected effects	Restore the resource store; epochs persist with it
Thundering herd on release	One contended lock	Acquire-rate spike and 409 burst	Jittered retries; queue admission	Predecessor-watch queue for that lock
Watch stream compacted	Waiters on affected locks	Compaction error on cursor	Recheck ownership through the authority	Snapshot and resume the watch

- Observe
 - Renewal tail latency and uncertain renewals.
 - Fence rejections at resources.
 - Contended lock duration and waiter count.
 - Authority quorum and clock health.
- Security
 - Do not accept a caller-supplied arbitrary epoch as proof of ownership.
 - Resource requests must carry an authenticated grant or arrive through a trusted ownership boundary.
- Retry discipline
 - Retry an acquisition timeout with the same requestId so replay returns the original epoch instead of allocating another.
 - Retry a renewal only until the conservative local deadline; then stop work regardless of the outcome.
 - Exponential backoff with jitter on 409 acquisition conflicts, or a predecessor watch when fairness matters.
 - A 410 expired lease is a corrective action: reacquire, do not retry the renewal.
- Tenant isolation
 - Bind lock names to authenticated tenant namespaces.
 - Quota per-tenant active leases, renewal rate, and waiter count so one tenant's herd cannot delay other tenants' renewals.
 - A tenant's token receives 404 on another tenant's lock names, not 403, to avoid existence leaks.

- Allocate epochs per scope so one tenant's token growth reveals nothing about another tenant's activity.
- Shard a lock-heavy tenant's namespace onto its own authority when renewal latency is at risk.
- Rollout and rollback safety
 - Deploy resource fencing before relying on new lock semantics.
 - Drain unfenced clients and reject their protocol after migration.
 - Roll the authority one replica at a time while preserving quorum; leases keep their deadlines across the roll.
 - A rollback must never lower the epoch allocator; the recovery epoch advances on every reopen.
 - Drill pause, partition, and snapshot-restore cases.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Lease plus resource fence	Expiring owner with monotonic epoch	Safe stale-writer rejection	Requires resource cooperation	Baseline distributed ownership
Database row lock	Transaction holds resource lock	Direct atomicity for DB-only work	Long transactions and connection cost	Short effect inside one database
Optimistic CAS only	Compare resource version at commit	No lease lifecycle	Conflicting work may be wasted	Small independent updates
Redis-style TTL lock	Expiring coordination key	Low coordination overhead	Failover and stale-effect limits	Best-effort coordination with explicit weaker guarantees

- Stronger requirement
 - Strict FIFO uses a durable waiter sequence and predecessor watches.
 - Cancellation and dead waiters need cleanup without skipping a live owner.
- At 10× load
 - Shard unrelated lock namespaces across authorities.
 - A single hot resource remains serial.
- Cost and blast radius
 - Sharing one session across many locks lowers renewal traffic but couples their loss.
- Migration
 - A new authority must preserve epoch ordering relative to existing resource fences.
- Weakest assumption
 - Every protected effect passes through an authenticated atomic fencing boundary.
- Over-engineering warning
 - Do not add distributed locks around a database update already protected by one CAS.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Ask whether the resource fences	Negotiate unsupported external effects	Ownership versus effect safety
Capacity	Size renewals and hot lock limit	Session grouping tradeoff	No lock-server cure for serialized work
Correctness	Conditional renew/release and activation	Recovery epoch continuity	Handles delayed messages and restore
Operations	Pause and partition tests	Migration from unfenced clients	Enforcement precedes guarantee
Architecture	Authority, client lease manager, protected resource, waiter queue, one trace	Cost of each part and where the fence physically lives	Every component has a mechanism and a price
Wrap	Recap with resource cooperation as the price	Names the unfenced-effect path as the weakest assumption	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: The old holder resumes after expiry?
 - **IC3 baseline**
 - The new holder activates a higher fence and the resource rejects the old epoch.
 - **IC4 stretch**
 - State that expiry alone does not update the resource fence.
- Q2: Can the current owner write twice?
 - **IC3 baseline**
 - Yes
 - Equal epoch is valid with distinct operation IDs.
 - **IC4 stretch**
 - Add per-resource versions if same-owner operations can arrive out of order.
- Q3: Why stop after an uncertain renewal?
 - **IC3 baseline**
 - The client cannot safely rely on a lease beyond its conservative deadline.
 - **IC4 stretch**
 - A resource grant-validation barrier can strengthen the contract at additional latency.
- Q4: A delayed release arrives after reacquisition?
 - **IC3 baseline**
 - Match the full owner, lease, and epoch tuple before deleting.
 - **IC4 stretch**
 - Retain idempotency results so acquisition replay does not allocate another epoch.
- Q5 (premise reversal): The resource cannot inspect tokens?
 - **IC3 baseline**
 - The lock cannot guarantee rejection of stale physical effects.
 - **IC4 stretch**
 - Choose destination idempotency, a transactional proxy, or an explicit weaker contract.
- Q6: Can an etcd revision be a fence forever?
 - **IC3 baseline**
 - Only while its ordering survives the authority's lifecycle.
 - **IC4 stretch**
 - A snapshot restore requires revision bumping or an independent recovery epoch.
- Q7 (premise reversal): Set the lease TTL to 2 s and skip fencing; does a short TTL bound the stale window enough?
 - **IC3 baseline**
 - A shorter TTL shrinks the window but never closes it; a GC pause or a delayed packet can exceed any TTL.
 - It also raises renewal load and false expiries, so availability drops without safety gained.
 - **IC4 stretch**
 - Treat TTL as a recovery-speed knob and fencing as the safety mechanism; tune them separately.
 - If fencing is impossible, state the residual duplicate window and the destination idempotency that absorbs it.
- Validation scenarios
 - Pause a holder beyond TTL, activate a new owner, then deliver the old write.
 - Apply two distinct operations at the same epoch successfully.
 - Deliver stale renew and release requests after reacquisition.
 - Lose a renewal response and check the client stops conservatively.
 - Resume a compacted watch through snapshot recovery.

- Restore older authority state while resource fences remain newer and reject unsafe grants.

Recall card

- "Lease expiry does not terminate the holder."
- "Fencing belongs at the protected effect."
- "Equal epoch is valid for the current owner's next operation."
- "Restore must preserve token monotonicity."
- Wrap-up script
 - "The lease authority grants one ordered owner and a monotonic epoch."
 - "The holder activates that epoch at the resource before protected work."
 - "Conditional renewals and atomic fence checks handle delayed or paused owners."
 - "The guarantee fails if an effect bypasses that enforcement boundary."

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): role decode and scoring context.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [etcd API guarantees](#): lease and consistency boundaries.
- [etcd disaster recovery](#): snapshot recovery and revision considerations.
- [Raft paper](#): quorum-backed ownership state.